

TASK MANAGEMENT WEB APPLICATION

‘TASK_FLOW’



MOHAMED IHSHAS

TRI/IT/2324/F/47

Higher National Diploma in Information Technology

SLIATE

Sri Lanka Institute of Advanced Technological Education (SLIATE)

Advanced Technological Institute – Trincomalee

Date Of Submission

2026

Declaration

I declare that this report is my own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is given. "

A.M.Ihshas

.....

Date :-

Supervised by

Thanushan

.....

Signature of Supervisor

Date :-

Acknowledgements

I would like to express my sincere gratitude to my supervisor for their valuable guidance, continuous support, and constructive feedback throughout this project.

I am also thankful to the Advanced Technological Institute for providing the resources and learning environment necessary for completing this project.

Finally, I extend my heartfelt appreciation to my family and friends for their encouragement and support during the development of this project.

A.M. IHSHAS

TRI | IT | 2324 | F | 47

TASKFLOW – TASK MANAGEMENT WEB APPLICATION

FINAL PROJECT REPORT

ABSTRACT

TaskFlow is a web-based task management application developed using HTML, CSS, and JavaScript. The system allows users to create, edit, delete, and manage daily tasks efficiently. The application uses browser LocalStorage to store task data in JSON format without requiring a backend database.

The system includes features such as task prioritization, due date management, search and filtering functionality, and dashboard statistics that display total tasks, completed tasks, and overdue tasks. The application is fully client-side and works offline.

TaskFlow demonstrates practical implementation of modern front-end technologies and client-side data management techniques.

Table of Contents

1. Introduction
 - 1.1 Overview of TaskFlow
 - 1.2 Objectives and Goals
2. Literature Review
 - 2.1 Overview of Existing Solutions
 - 2.2 Key Challenges in Current Systems
 - 2.3 Technologies Relevant to TaskFlow
3. Project Methodology
 - 3.1 Tools and Technologies Used
 - 3.2 Development Process
 - 3.3 System Architecture
 - 3.4 Implementation Process
4. Results and Discussion
 - 4.1 System Functionality
 - 4.2 User Interface and Design
 - 4.3 Testing and Evaluation
5. Risk Analysis
6. Gantt Chart Explanation
7. Conclusion
8. References
9. Appendices

List of Abbreviations

HTML – HyperText Markup Language

CSS – Cascading Style Sheets

JS – JavaScript

IT – Information Technology

HNDIT – Higher National Diploma in Information Technology

ATI – Advanced Technological Institute

Chapter 1: Introduction

1.1 Overview of Task_Flow

Task_Flow is a single-user task management web application designed to help individuals organize their daily activities effectively. The system allows users to create tasks, assign priorities, set due dates, update status, and track progress.

Unlike cloud-based systems, TaskFlow operates entirely in the browser using LocalStorage, eliminating the need for a backend server or database.

1.2 Objectives and Goals

- Provide a simple and intuitive task management system.
- Allow users to add, edit, and delete tasks.
- Enable task priority selection (Low, Medium, High).
- Provide due date tracking and overdue detection.
- Implement search and filtering features.
- Display dashboard statistics.
- Ensure offline functionality.

TaskFlow aims to improve personal productivity through structured digital task organization.

Chapter 2: Literature Review

2.1 Overview of Existing Solutions

Many modern task management systems exist, including:

- Todoist
- Microsoft to Do
- Google Tasks
- Trello

These platforms rely on cloud servers and databases to store user data. They provide synchronization across devices but require internet connectivity and user authentication.

2.2 Key Challenges in Current Systems

- Require internet connection
- Depend on backend infrastructure
- May involve subscription costs
- Complex setup for beginners

TaskFlow addresses these challenges by providing a lightweight browser-based solution.

2.3 Technologies Relevant to TaskFlow

Frontend Technologies

HTML – Defines structure

CSS – Controls design and layout

JavaScript – Implements logic and interactivity

Web Storage API

LocalStorage allows persistent client-side storage in JSON format.

Chapter 3: Project Methodology

3.1 Tools and Technologies Used

Frontend: HTML, CSS, JavaScript

Storage: LocalStorage

Development Tool: Visual Studio Code was the primary tool used for writing and editing the HTML and CSS code.



Browser Testing: Chrome, Edge, Firefox

3.2 Development Process

Planning – Defined system requirements and interface design.

Design – Created responsive UI layout and dashboard structure.

Development – Implemented CRUD operations and LocalStorage integration.

Testing – Tested across browsers and devices.

3.3 System Architecture

TaskFlow follows a client-side architecture:

Presentation Layer – HTML + CSS

Logic Layer – JavaScript

Storage Layer – LocalStorage

No backend or database is required.

3.4 Implementation Process

- Created structured HTML layout.
 - Designed user interface using CSS.
 - Implemented task management logic in JavaScript.
 - Integrated LocalStorage for data persistence.
 - Tested and debugged application.
-

Chapter 4: Results and Discussion

4.1 System Functionality

The system successfully allows:

- Task creation
 - Task editing
 - Task deletion
 - Status update
 - Priority assignment
 - Overdue detection
 - Dashboard statistics
-

4.2 User Interface and Design

The interface is clean, responsive, and user-friendly.

It includes:

- Dashboard cards
- Task input form
- Task list display
- Modal edit window

The design ensures smooth user interaction.

4.3 Testing and Evaluation

The application was tested for:

- Add task functionality
- Edit task functionality
- Delete task functionality
- Data persistence after refresh
- Filter and search operations

All tests were successful.

Chapter 5: Risk Analysis

Data Loss – If browser data is cleared, tasks will be lost.

Mitigation: Inform users and plan export feature.

Browser Compatibility – Different browser behaviors.

Mitigation: Cross-browser testing.

Development Delay – Time management issues.

Mitigation: Follow project schedule.

Chapter 6: Gantt Chart Explanation

The project development was divided into phases:

Requirement Analysis – Week 1

Design – Week 2

Development – Week 3–4

Testing – Week 5

Documentation – Week 6

The Gantt chart ensured proper scheduling and progress tracking.

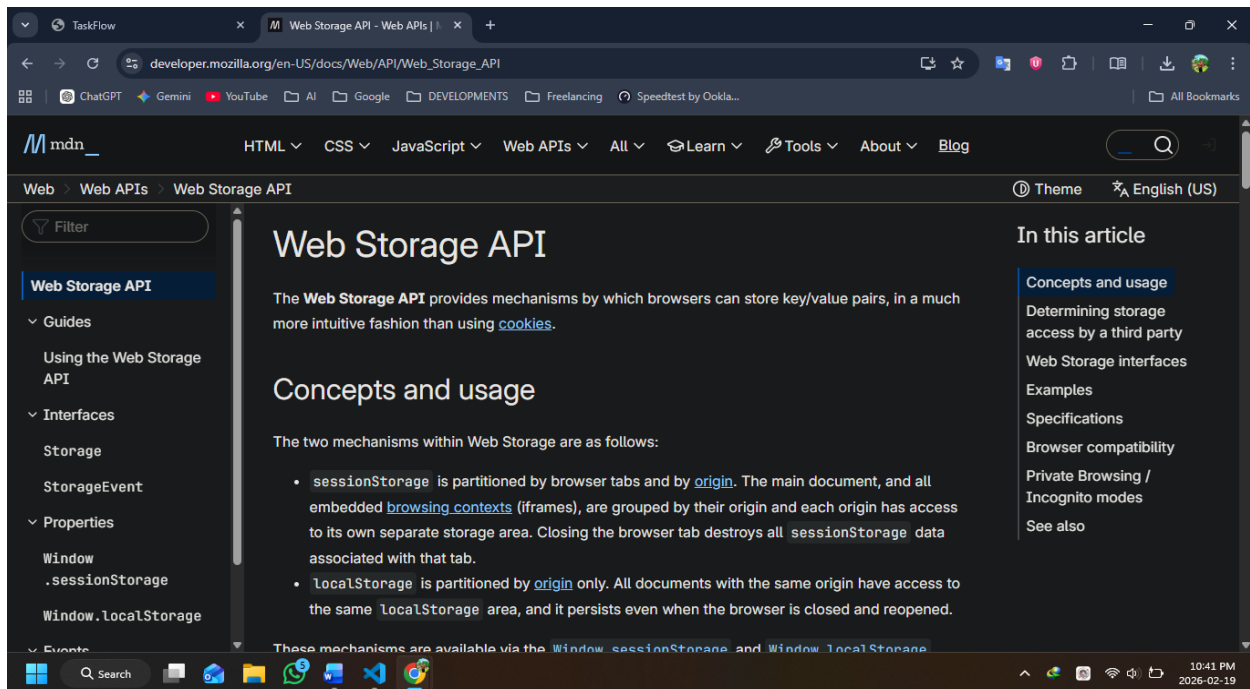
Chapter 7: Conclusion

TaskFlow was successfully developed as a lightweight, browser-based task management system. The application meets all defined objectives and demonstrates effective use of front-end technologies.

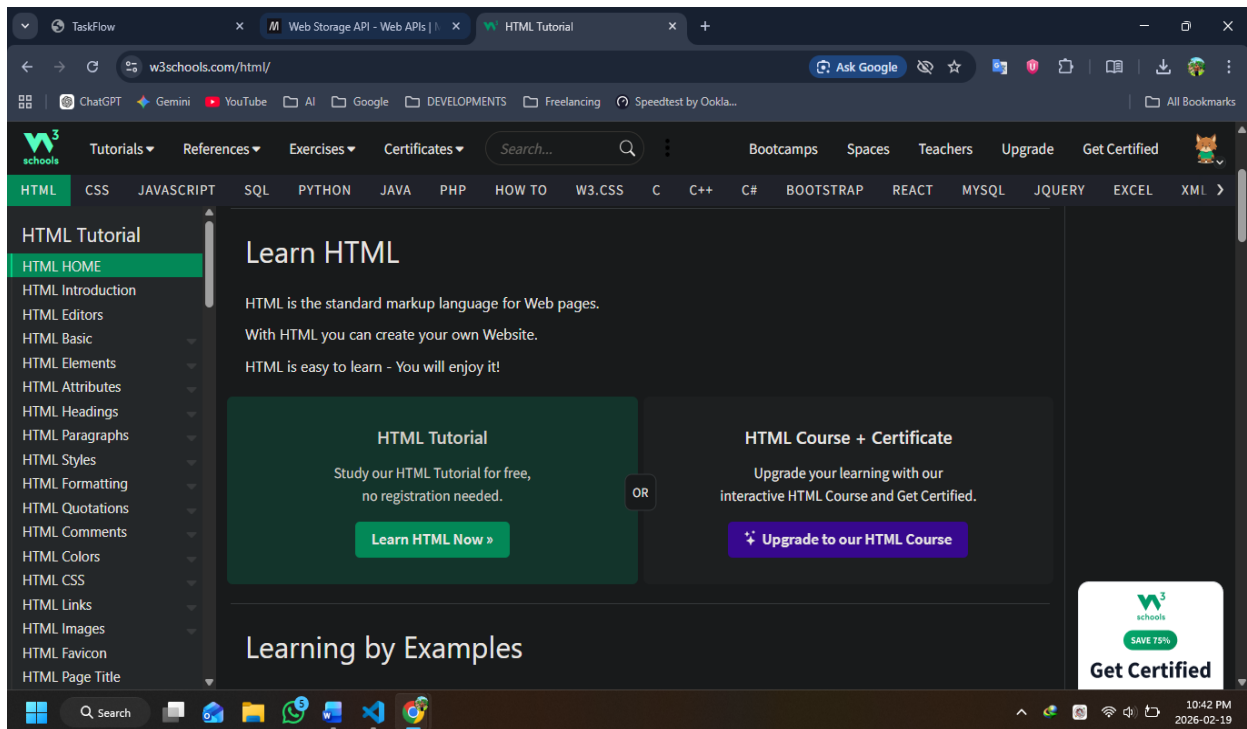
The project proves that a fully functional web application can be developed without backend infrastructure.

Chapter 8: References

MDN Web Docs – Web Storage API

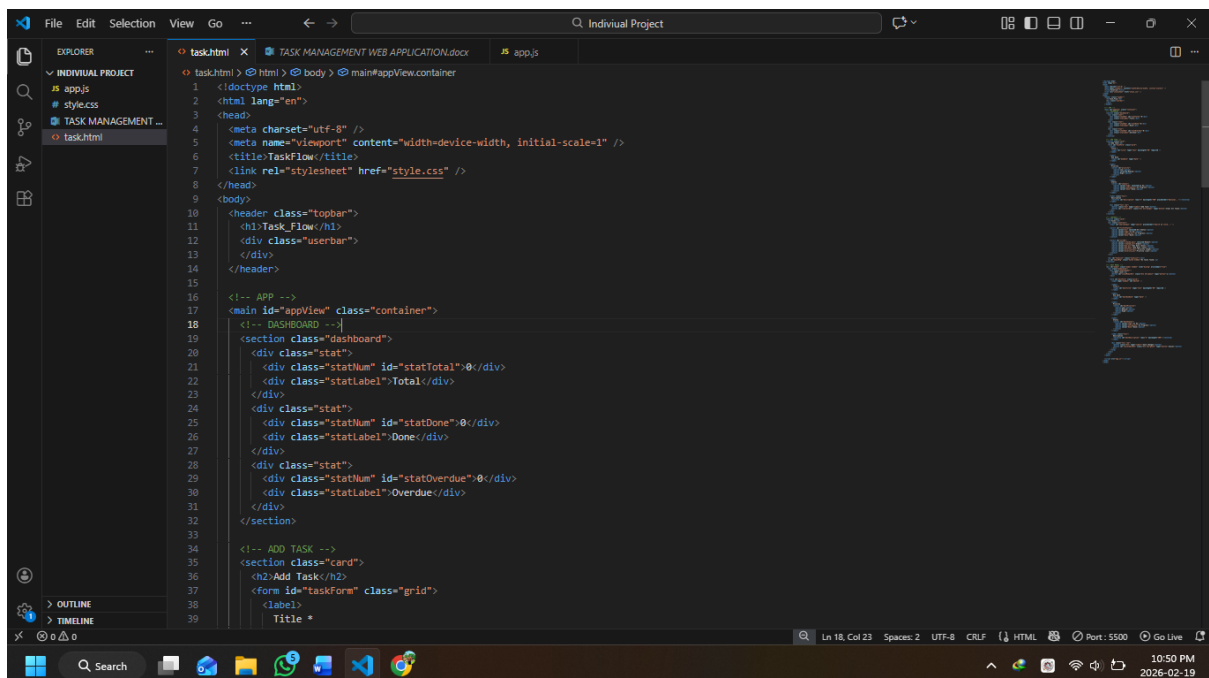


W3Schools – HTML, CSS, JavaScript



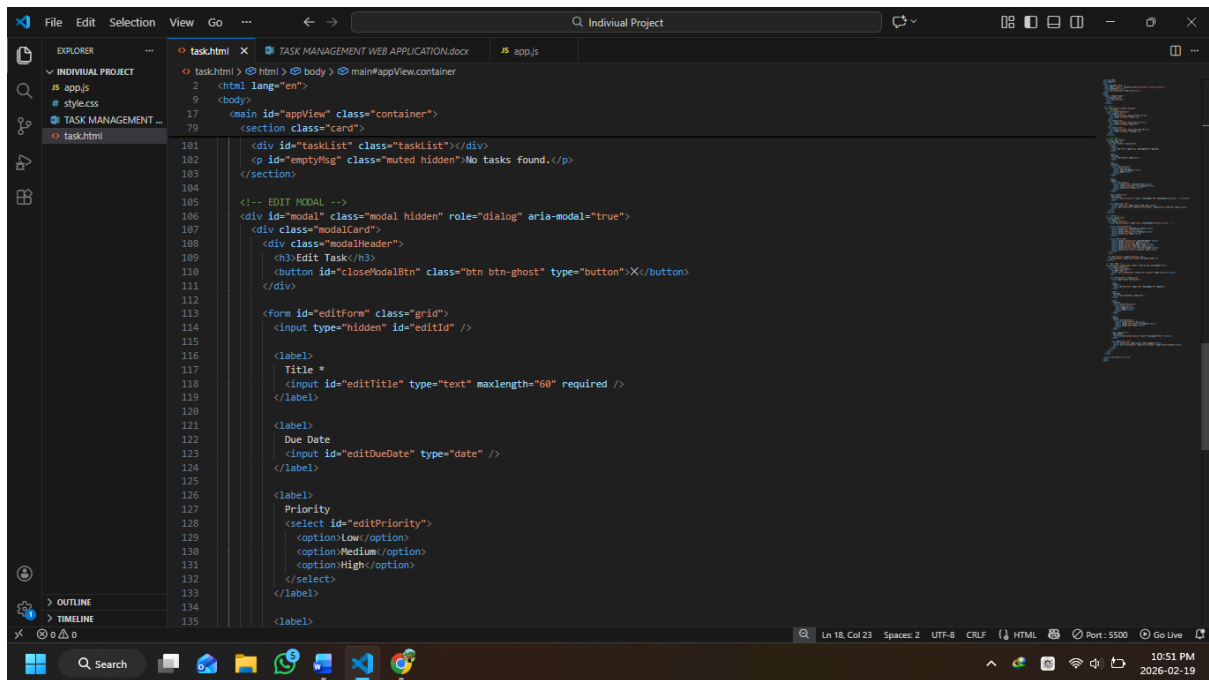
Chapter 9: Appendices

- HTML Code

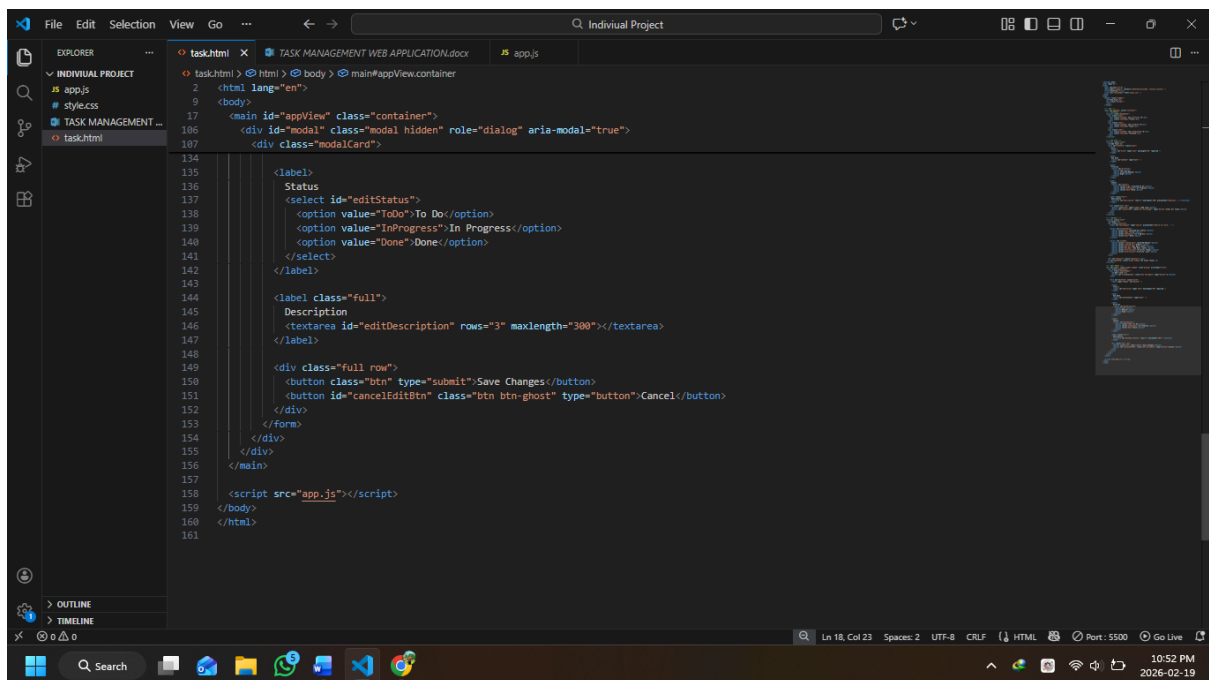


```
File Edit Selection View Go ... Individual Project
taskhtml X TASK MANAGEMENT WEB APPLICATION.docx app.js
EXPLORER
INDIVIDUAL PROJECT
app.js
style.css
TASK MANAGEMENT ...
taskhtml
OUTLINE
TIMELINE
0
2 <html lang="en">
3 <body>
4 <main id="appView" class="container">
5 <section class="card">
6 <form id="taskForm" class="grid">
7
8   <label>
9     Title *
10    <input id="title" type="text" maxlength="60" required />
11  </label>
12
13  <label>
14    Due Date
15    <input id="dueDate" type="date" />
16  </label>
17
18  <label>
19    Priority
20    <select id="priority">
21      <option>Low</option>
22      <option selected="selected">Medium</option>
23      <option>High</option>
24    </select>
25  </label>
26
27  <label>
28    Status
29    <select id="status">
30      <option value="ToDo" selected="selected">To Do</option>
31      <option value="InProgress">In Progress</option>
32      <option value="Done">Done</option>
33    </select>
34  </label>
35
36  <label class="full">
37    Description
38    <textarea id="description" rows="3" maxlength="300" placeholder="Optional..."></textarea>
39  </label>
40
41  <div class="full row">
42    <button class="btn" type="submit">Add Task</button>
43  </div>
44 </form>
45 </section>
46 </main>
47 </body>
48 </html>
```

```
File Edit Selection View Go ... Individual Project
taskhtml X TASK MANAGEMENT WEB APPLICATION.docx app.js
EXPLORER
INDIVIDUAL PROJECT
app.js
style.css
TASK MANAGEMENT ...
taskhtml
OUTLINE
TIMELINE
0
70 </div>
71 <div class="full row">
72   <button class="btn" type="submit">Add Task</button>
73   <button id="clearAllBtn" class="btn btn-danger" type="button">Clear All Tasks</button>
74 </div>
75 </form>
76 </section>
77 </main>
78 </body>
79 </html>
80 <!-- CONTROLS -->
81 <section class="card">
82   <h2>Tasks</h2>
83   <div class="controls">
84     <input id="searchInput" type="search" placeholder="Search by title..." />
85
86     <select id="filterStatus">
87       <option value="All" selected="selected">All Status</option>
88       <option value="ToDo">To Do</option>
89       <option value="InProgress">In Progress</option>
90       <option value="Done">Done</option>
91     </select>
92
93     <select id="sortBy">
94       <option value="created_desc" selected="selected">Newest</option>
95       <option value="created_asc">Oldest</option>
96       <option value="due_asc">Due Date (Soon)</option>
97       <option value="due_desc">Due Date (Late)</option>
98       <option value="priority_desc">Priority (High)</option>
99       <option value="priority_asc">Priority (Low)</option>
100    </select>
101  </div>
102  <div id="taskList" class="taskList"></div>
103  <p id="emptyMsg" class="muted hidden">No tasks found.</p>
104 </section>
```



```
1 <html>
2   <html lang="en">
3     <body>
4       <main id="appView" class="container">
5         <section class="card">
6           <div id="taskList" class="taskList"></div>
7           <p id="emptyMsg" class="muted hidden">No tasks found.</p>
8         </section>
9
10        <!-- EDIT MODAL -->
11        <div id="modal" class="modal hidden" role="dialog" aria-modal="true">
12          <div class="modalCard">
13            <div class="modalHeader">
14              <h3>Edit Task</h3>
15              <button id="closeModalBtn" class="btn btn-ghost" type="button">X</button>
16            </div>
17
18            <form id="editForm" class="grid">
19              <input type="hidden" id="editId" />
20
21              <label>
22                Title *
23                <input id="editTitle" type="text" maxlength="60" required />
24              </label>
25
26              <label>
27                Due Date
28                <input id="editDueDate" type="date" />
29              </label>
30
31              <label>
32                Priority
33                <select id="editPriority">
34                  <option>Low</option>
35                  <option>Medium</option>
36                  <option>High</option>
37                </select>
38              </label>
39
40            </form>
41          </div>
42        </div>
43      </main>
44    </body>
45  </html>
```



```
134 <div id="modal" class="modal hidden" role="dialog" aria-modal="true">
135   <div class="modalCard">
136     <div class="modalHeader">
137       <h3>Edit Task</h3>
138       <button id="closeModalBtn" class="btn btn-ghost" type="button">X</button>
139     </div>
140
141     <form id="editForm" class="grid">
142       <input type="hidden" id="editId" />
143
144       <label>
145         Status
146         <select id="editStatus">
147           <option value="To Do">To Do</option>
148           <option value="In Progress">In Progress</option>
149           <option value="Done">Done</option>
150         </select>
151       </label>
152
153       <label class="full">
154         Description
155         <textarea id="editDescription" rows="3" maxlength="300"></textarea>
156       </label>
157
158       <div class="full row">
159         <button class="btn" type="submit">Save Changes</button>
160         <button id="cancelEditBtn" class="btn btn-ghost" type="button">Cancel</button>
161       </div>
162     </form>
163   </div>
164 </div>
165 </main>
166 </body>
167 </html>
```

- CSS Code

```
# style.css
1 :root{
2   --bg:#0a111e;
3   --card:#121b2f;
4   --muted:#9fb0d0;
5   --text:#e9f0ff;
6   --line:#0d152b;
7   --accent:#eaf1ff;
8   --danger:#ff4d4d;
9 }
10
11 *{box-sizing:border-box}
12 body{
13   margin:0;
14   font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
15   background: radial-gradient(1200px 600px at 20% 0%, #16254a 0%, var(--bg) 55%);
16   color:var(--text);
17 }
18
19 .topbar{
20   display:flex;
21   justify-content:space-between;
22   align-items:center;
23   padding:14px 18px;
24   border-bottom:1px solid var(--line);
25   position:sticky;
26   top:0;
27   background: rgba(11,18,32,.85);
28   backdrop-filter: blur(10px);
29 }
30
31 h1{margin:0;font-size:18px;letter-spacing:.5px}
32 .userbar{display:flex;gap:10px;align-items:center}
33 #helloUser{color:var(--muted);font-size:14px}
34
35 .container{max-width:980px;margin:18px auto;padding:0 14px}
36 .card{
37   max-width:980px;
38   margin:18px auto;
39   padding:16px;
```

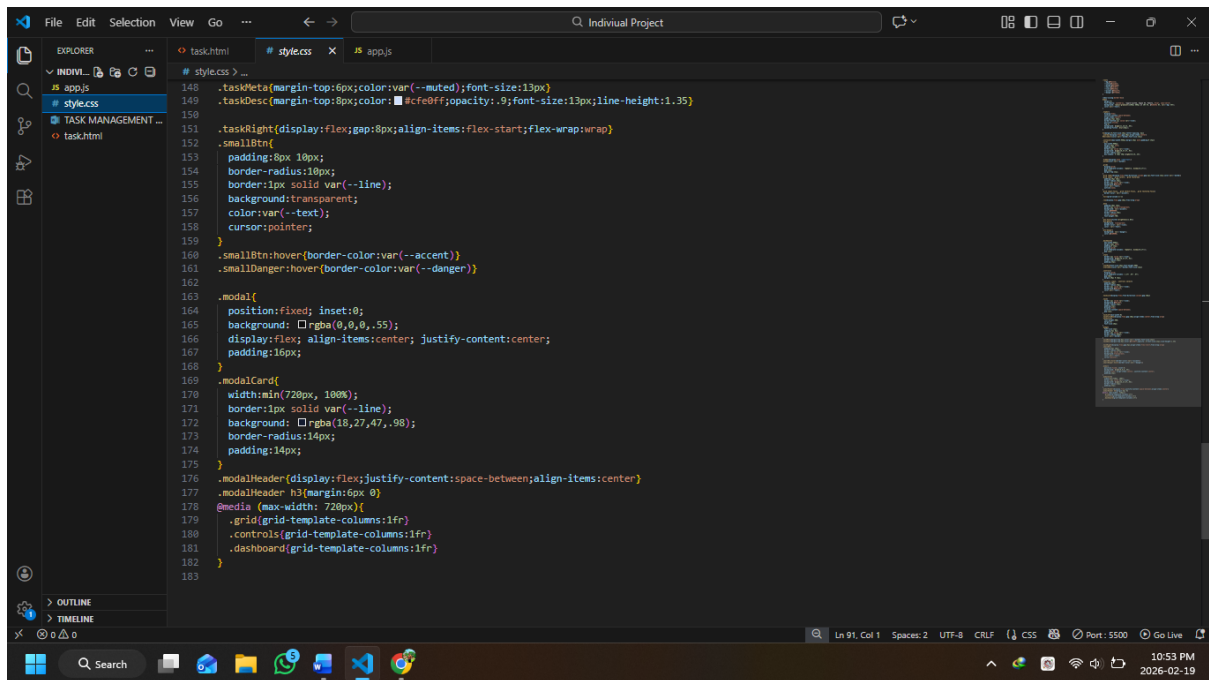
```
# style.css
35 .container{max-width:980px;margin:18px auto;padding:0 14px}
36 .card{
37   max-width:980px;
38   margin:18px auto;
39   padding:16px;
40   border:1px solid var(--line);
41   background: rgba(18,27,47,.92);
42   border-radius:16px;
43   box-shadow: 0 10px 30px rgba(0,0,0,.25);
44 }
45
46 .hidden{display:none !important;}
47 .muted{color:var(--muted)}
48
49 .grid{
50   display:grid;
51   grid-template-columns: repeat(2, minmax(0,1fr));
52   gap:12px;
53   margin-top:12px;
54 }
55
56 .grid label{display:flex;flex-direction:column;gap:1px;font-size:13px;color:var(--muted)}
57 .grid input, .grid select, .grid textarea{
58   padding:10px 10px;
59   border-radius:10px;
60   border:1px solid var(--line);
61   background:#0c1427;
62   color:var(--text);
63   outline:none;
64 }
65
66 .grid input:focus, .grid select:focus, .grid textarea:focus{
67   border-color: var(--accent);
68 }
69
70 .full{grid-column:1/-1}
71
72 .row{display:flex;gap:10px;flex-wrap:wrap}
73
74 .btn{
75   padding:10px 14px;
76   border:1px solid transparent;
```

This screenshot shows the VS Code editor with a project named 'Individual Project'. The Explorer sidebar on the left shows the file structure with 'style.css' selected. The main editor area displays CSS code for buttons and a dashboard layout. The code includes styles for a button, a button hover state, a button danger state, a dashboard container, a stat card, and a controls section.

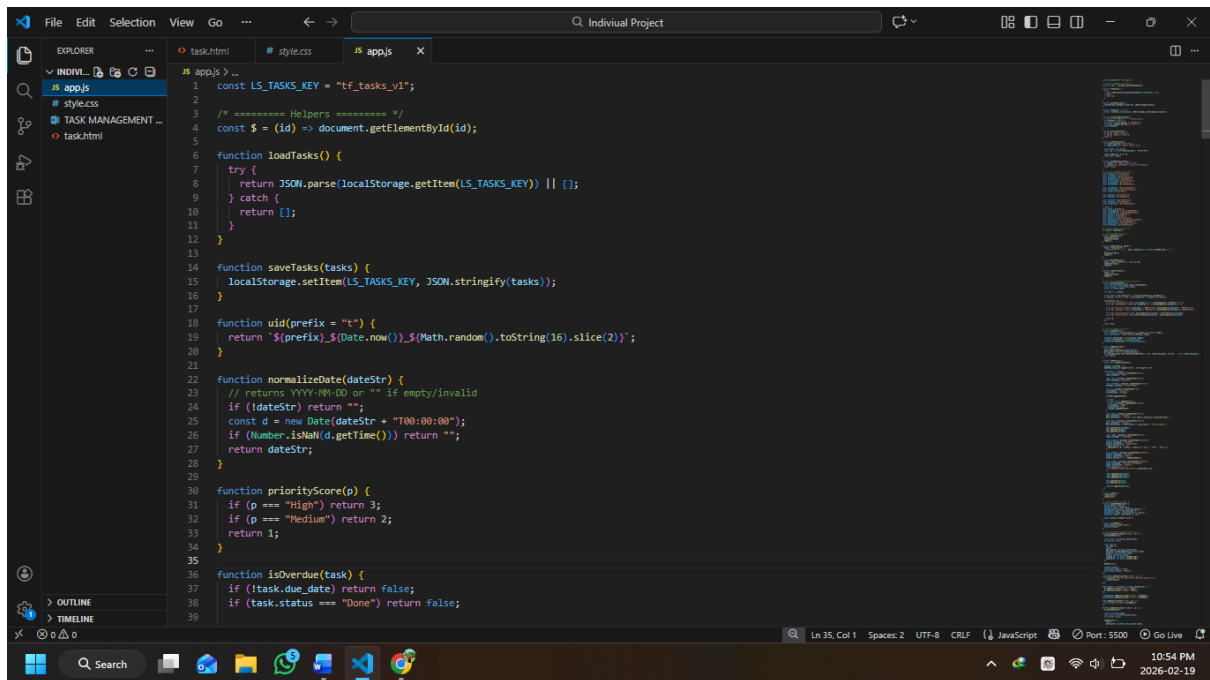
```
71 .btn{
72   padding:10px 14px;
73   border:1px solid transparent;
74   background: var(--accent);
75   color: #000000;
76   border-radius:10px;
77   cursor:pointer;
78   font-weight:700;
79 }
80 .btn:hover{filter:brightness(1.05)}
81 .btn-host{
82   background: transparent;
83   border-color: var(--line);
84   color: var(--text);
85 }
86 .btn-danger{
87   background: var(--danger);
88   color: #100000;
89 }
90
91
92 .dashboard{
93   max-width:980px;
94   margin:18px auto;
95   padding:0 14px;
96   display:grid;
97   grid-template-columns: repeat(3, minmax(0,1fr));
98   gap:12px;
99 }
100
101 .stat{
102   border:1px solid var(--line);
103   background: #181818;
104   border-radius:14px;
105   padding:14px;
106 }
107 .statNum{font-size:26px;font-weight:900}
108 .statLabel{color:var(--muted);font-size:13px}
109
110 .controls{
```

This screenshot shows the VS Code editor with the same project. The main editor area displays CSS code for controls, a task list, and a badge. The code includes styles for controls input and select, a task list container, a task left element, a task title row, a task title, and a badge.

```
109 .controls{
110   display:grid;
111   grid-template-columns: 1.2fr .8fr .8fr;
112   gap:10px;
113   margin:10px 0 14px;
114 }
115 .controls input, .controls select{
116   padding:10px;
117   border-radius:10px;
118   border:1px solid var(--line);
119   background: #000000;
120   color:var(--text);
121 }
122
123 .taskList{display:flex;flex-direction:column;gap:10px}
124
125 .task{
126   border:1px solid var(--line);
127   background: #000000;
128   border-radius:14px;
129   padding:12px;
130   display:flex;
131   justify-content:space-between;
132   gap:12px;
133 }
134 .taskLeft(min-width:0)
135 .taskTitleRow{display:flex;gap:10px;align-items:center;flex-wrap:wrap}
136 .taskTitle{
137   font-weight:900;
138   margin:0;
139   font-size:16px;
140 }
141 .badge{
142   font-size:12px;
143   padding:4px 8px;
144   border:1px solid var(--line);
145   border-radius:999px;
146   color:var(--muted);
147 }
```

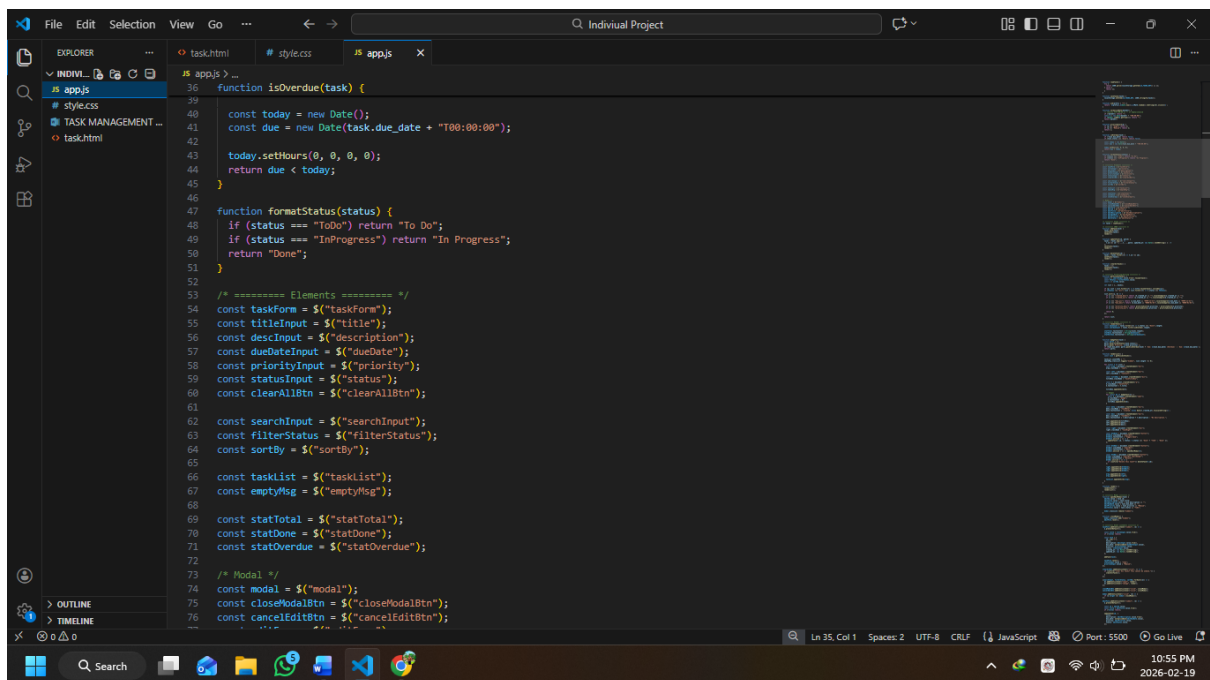


- JavaScript Code



The screenshot shows the Visual Studio Code editor with the 'app.js' file open. The code defines several utility functions for a task management application:

```
1 const LS_TASKS_KEY = "tf_tasks_v1";
2
3 /* ===== Helpers ===== */
4 const $ = (id) => document.getElementById(id);
5
6 function loadTasks() {
7   try {
8     return JSON.parse(localStorage.getItem(LS_TASKS_KEY)) || [];
9   } catch {
10     return [];
11   }
12 }
13
14 function saveTasks(tasks) {
15   localStorage.setItem(LS_TASKS_KEY, JSON.stringify(tasks));
16 }
17
18 function uid(prefix = "t") {
19   return `${prefix}-${Date.now()}-${Math.random().toString(16).slice(2)}`;
20 }
21
22 function normalizeDate(dateStr) {
23   // returns YYYY-MM-DD or "" if empty/invalid
24   if (!dateStr) return "";
25   const d = new Date(dateStr + "T00:00:00");
26   if (Number.isNaN(d.getTime())) return "";
27   return dateStr;
28 }
29
30 function priorityScore(p) {
31   if (p === "High") return 3;
32   if (p === "Medium") return 2;
33   return 1;
34 }
35
36 function isOverdue(task) {
37   if (!task.due_date) return false;
38   if (task.status === "Done") return false;
```



The screenshot shows the Visual Studio Code editor with the 'app.js' file open, displaying the continuation of the JavaScript code:

```
36 function isOverdue(task) {
39   const today = new Date();
40   const due = new Date(task.due_date + "T00:00:00");
41   today.setHours(0, 0, 0);
42   return due < today;
43 }
44
45 function formatStatus(status) {
46   if (status === "ToDo") return "To Do";
47   if (status === "InProgress") return "In Progress";
48   return "Done";
49 }
50
51 /* ===== Elements ===== */
52
53 const taskForm = $("#taskForm");
54 const titleInput = $("#title");
55 const descInput = $("#description");
56 const dueDateInput = $("#dueDate");
57 const priorityInput = $("#priority");
58 const statusInput = $("#status");
59 const clearAllBtn = $("#clearAllBtn");
60
61 const searchInput = $("#searchInput");
62 const filterStatus = $("#filterStatus");
63 const sortBy = $("#sortBy");
64
65 const taskList = $("#taskList");
66 const emptyMsg = $("#emptyMsg");
67
68 const statTotal = $("#statTotal");
69 const statDone = $("#statDone");
70 const statOverdue = $("#statOverdue");
71
72 /* Modal */
73 const modal = $("#modal");
74 const closeModalBtn = $("#closeModalBtn");
75 const cancelEditBtn = $("#cancelEditBtn");
```

This screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure with folders for 'taskhtml', 'style.css', and 'app.js'. The main editor window is open to 'app.js', showing JavaScript code for a task management application. The code includes a 'Model' section with constants for form elements, a 'State' section with a 'tasks' array, and three main functions: 'addTask', 'updateTask', and 'deleteTask'. The 'addTask' function pushes a new task to the array and saves it. The 'updateTask' function finds a task by ID and updates its details. The 'deleteTask' function removes a task from the array. The code is written in a dark theme with syntax highlighting.

```
73  /* Model */
74  const modal = $("#modal");
75  const closeModalBtn = $("#closeModalBtn");
76  const cancelEditBtn = $("#cancelEditBtn");
77  const editForm = $("#editForm");
78  const editId = $("#editId");
79  const editTitle = $("#editTitle");
80  const editDescription = $("#editDescription");
81  const editDueDate = $("#editDueDate");
82  const editPriority = $("#editPriority");
83  const editStatus = $("#editStatus");
84
85  /* ===== State ===== */
86  let tasks = loadTasks();
87
88  /* ===== CRUD ===== */
89  function addTask(task) {
90    tasks.push(task);
91    saveTasks(tasks);
92    render();
93  }
94
95  function updateTask(id, patch) {
96    tasks = tasks.map((t) => {
97      t.id === id ? { ...t, ...patch, updated_at: new Date().toISOString() } : t
98    });
99    saveTasks(tasks);
100    render();
101  }
102
103  function deleteTask(id) {
104    tasks = tasks.filter((t) => t.id !== id);
105    saveTasks(tasks);
106    render();
107  }
108
109  function clearAllTasks() {
110    tasks = [];
111    saveTasks(tasks);
```

This screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure with folders for 'taskhtml', 'style.css', and 'app.js'. The main editor window is open to 'app.js', showing JavaScript code for a task management application. The code includes a 'Filtering/Sorting' section with a 'getVisibleTasks' function that filters tasks based on search input, status, and sorting criteria. It also includes a 'Render' section with a 'renderStats' function that updates the UI with task counts and a 'badgeText' function that generates badge text for tasks. The code is written in a dark theme with syntax highlighting.

```
114
115  /* ===== Filtering/Sorting ===== */
116  function getVisibleTasks() {
117    const q = searchInput.value.trim().toLowerCase();
118    const fStatus = filterStatus.value;
119    const s = sortBy.value;
120
121    let list = [...tasks];
122
123    if (q) list = list.filter((t) => t.title.toLowerCase().includes(q));
124    if (fStatus !== "All") list = list.filter((t) => t.status === fStatus);
125
126    list.sort((a, b) => {
127      if (s === "created_desc") return (b.created_at || "").localeCompare(a.created_at || "");
128      if (s === "created_asc") return (a.created_at || "").localeCompare(b.created_at || "");
129
130      if (s === "due_desc") return (a.due_date || "9999-12-31").localeCompare(b.due_date || "9999-12-31");
131      if (s === "due_asc") return (b.due_date || "0000-01-01").localeCompare(a.due_date || "0000-01-01");
132
133      if (s === "priority_desc") return priorityScore(b.priority) - priorityScore(a.priority);
134      if (s === "priority_asc") return priorityScore(a.priority) - priorityScore(b.priority);
135
136      return 0;
137    });
138
139    return list;
140  }
141
142  /* ===== Render ===== */
143  function renderStats() {
144    const doneCount = tasks.filter((t) => t.status === "Done").length;
145    const overdueCount = tasks.filter(isOverdue).length;
146
147    statTotal.textContent = String(tasks.length);
148    statDone.textContent = String(doneCount);
149    statOverdue.textContent = String(overdueCount);
150  }
151
152  function badgeText(task) {
```

This screenshot shows the Visual Studio Code editor with the 'app.js' file open. The Explorer sidebar on the left shows the project structure with 'task.html', 'style.css', and 'app.js'. The main editor area displays the following JavaScript code:

```
152 function badgeText(task) {
153   const parts = [];
154   parts.push(formatStatus(task.status));
155   parts.push("Priority: " + task.priority);
156   if (task.due_date) parts.push(task.due_date ? "Due: " + task.due_date : "Due: " + task.due_date);
157   return parts;
158 }
159
160 function renderList() {
161   const list = getVisibleTasks();
162
163   taskList.innerHTML = "";
164   emptyMsg.classList.toggle("hidden", list.length !== 0);
165
166   for (const t of list) {
167     const wrap = document.createElement("div");
168     wrap.className = "task";
169
170     const left = document.createElement("div");
171     left.className = "taskLeft";
172
173     const titleRow = document.createElement("div");
174     titleRow.className = "taskTitleRow";
175
176     const h = document.createElement("p");
177     h.className = "taskTitle";
178     h.textContent = t.title;
179     titleRow.appendChild(h);
180
181     // Badges
182     for (const bt of badgeText(t)) {
183       const b = document.createElement("span");
184       b.className = "badge";
185       b.textContent = bt;
186       titleRow.appendChild(b);
187     }
188
189     const meta = document.createElement("div");
```

The status bar at the bottom indicates the cursor is at Line 35, Column 1, in a JavaScript file using UTF-8 encoding and CRLF line endings. The system clock shows 10:55 PM on 2026-02-19.

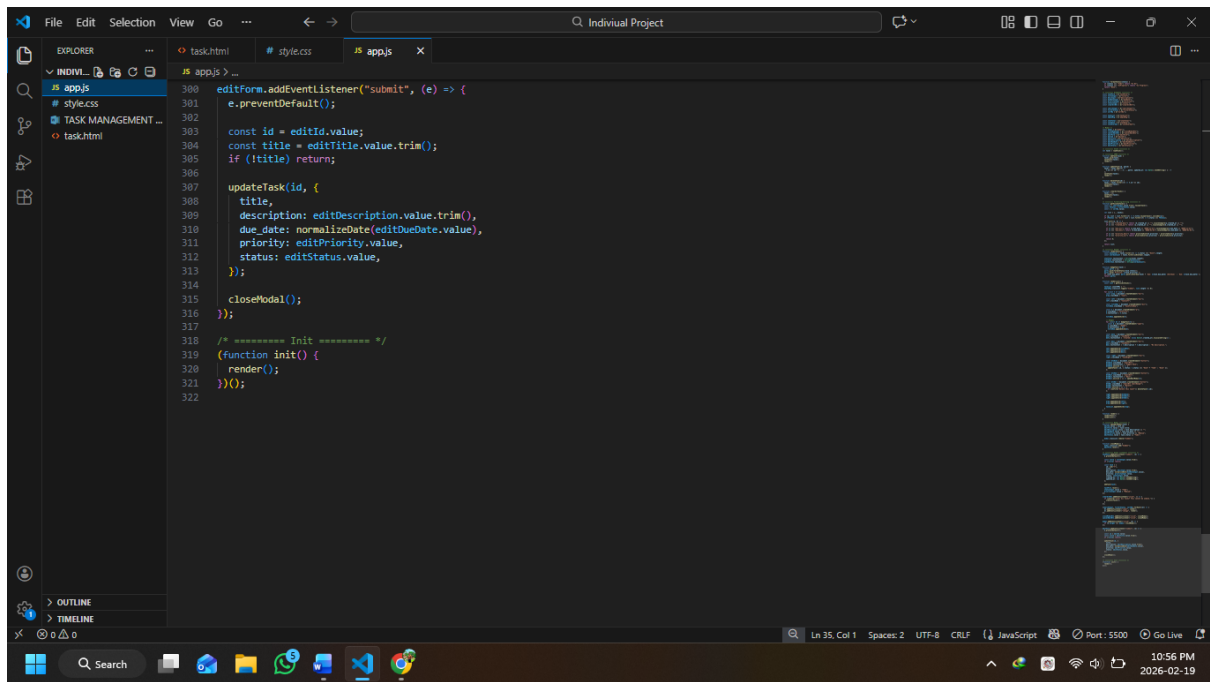
This screenshot shows the Visual Studio Code editor with the 'app.js' file open, displaying the continuation of the 'renderList' function. The Explorer sidebar on the left shows the project structure with 'task.html', 'style.css', and 'app.js'. The main editor area displays the following JavaScript code:

```
160 function renderList() {
161   const meta = document.createElement("div");
162   meta.className = "taskMeta";
163   meta.textContent = "Created: " + new Date(t.created_at).toLocaleString();
164
165   const desc = document.createElement("div");
166   desc.className = "taskDesc";
167   desc.textContent = t.description ? t.description : "No description.";
168
169   left.appendChild(titleRow);
170   left.appendChild(meta);
171   left.appendChild(desc);
172
173   const right = document.createElement("div");
174   right.className = "taskRight";
175
176   const btnDone = document.createElement("button");
177   btnDone.className = "smallBtn";
178   btnDone.textContent = "Toggle Done";
179   btnDone.onclick = () => {
180     updateTask(t.id, { status: t.status === "Done" ? "ToDo" : "Done" });
181   };
182
183   const btnEdit = document.createElement("button");
184   btnEdit.className = "smallBtn";
185   btnEdit.textContent = "Edit";
186   btnEdit.onclick = () => openEditModal(t);
187
188   const btnDel = document.createElement("button");
189   btnDel.className = "smallBtn smallDanger";
190   btnDel.textContent = "Delete";
191   btnDel.onclick = () => {
192     if (confirm("Delete this task?")) deleteTask(t.id);
193   };
194
195   right.appendChild(btnDone);
196   right.appendChild(btnEdit);
197   right.appendChild(btnDel);
```

The status bar at the bottom indicates the cursor is at Line 35, Column 1, in a JavaScript file using UTF-8 encoding and CRLF line endings. The system clock shows 10:56 PM on 2026-02-19.

```
File Edit Selection View Go ... Individual Project
EXPLORER task.html # style.css app.js x
# app.js
# style.css
TASK MANAGEMENT ...
task.html
160 function renderList() {
161
162     wrap.appendChild(left);
163     wrap.appendChild(right);
164     taskList.appendChild(wrap);
165 }
166
167 function render() {
168     renderStats();
169     renderList();
170 }
171
172 /* ===== Modal ===== */
173 function openEditModal(task) {
174     editId.value = task.id;
175     editTitle.value = task.title;
176     editDescription.value = task.description || "";
177     editDueDate.value = task.due_date || "";
178     editPriority.value = task.priority || "Medium";
179     editStatus.value = task.status || "ToDo";
180     modal.classList.remove("hidden");
181 }
182
183 function closeModal() {
184     modal.classList.add("hidden");
185     editForm.reset();
186 }
187
188 /* ===== Event Listeners ===== */
189 taskForm.addEventListener("submit", (e) => {
190     e.preventDefault();
191
192     const title = titleInput.value.trim();
193     if (!title) return;
194     const task = {
```

```
File Edit Selection View Go ... Individual Project
EXPLORER task.html # style.css app.js x
# app.js
# style.css
TASK MANAGEMENT ...
task.html
258 taskForm.addEventListener("submit", (e) => {
259
260     const task = {
261         id: uid("t"),
262         title,
263         description: descInput.value.trim(),
264         due_date: normalizeDate(dueDateInput.value),
265         priority: priorityInput.value,
266         status: statusInput.value,
267         created_at: new Date().toISOString(),
268         updated_at: new Date().toISOString(),
269     };
270
271     addTask(task);
272
273     taskForm.reset();
274     statusInput.value = "ToDo";
275     priorityInput.value = "Medium";
276 });
277
278 clearAllBtn.addEventListener("click", () => {
279     if (confirm("Clear ALL tasks? This cannot be undone. ")) {
280         clearAllTasks();
281     }
282 });
283
284 [searchInput, filterStatus, sortBy].forEach((el) => {
285     el.addEventListener("input", render);
286     el.addEventListener("change", render);
287 });
288
289 closeModalBtn.addEventListener("click", closeModal);
290 cancelEditBtn.addEventListener("click", closeModal);
291
292 modal.addEventListener("click", (e) => {
293     if (e.target === modal) closeModal();
294 });
295
296 editForm.addEventListener("submit", (e) => {
```



Project Web Page

